



Exception Handling



What will be covered

- throw keyword
- Use defined exceptions
- Throws keyword
- Throws example
- Rethrowing exception
- Improved catch block
- Try with resources
- Advantages of exception handling

throw keyword

- By default, when an exception condition occurs the system automatically throw an exception to inform user that there is something wrong. However we can also throw exception explicitly based on our own defined condition.
- Using “**throw keyword**” we can throw **checked, unchecked** and **user-defined exceptions**.
- Only object of Throwable class or its sub classes can be thrown.
- Program execution stops on encountering **throw** statement, and the closest catch statement is checked for matching type of exception.

- Syntax :

```
throw ThrowableInstance
```

throw keyword

OBJECT

```
public Object pop() {  
    Object obj;  
  
    if (size == 0) {  
        throw new EmptyStackException();  
    }  
  
    obj = objectAt(size - 1);  
    setObjectAt(size - 1, null);  
    size--;  
    return obj;  
}
```

- If the exception thrown is checked exception, handling using try-catch block is compulsory and if it is unchecked exception, try-catch block is optional

- Most of the times when we are developing an application in java, we often feel a need to create and throw **our own exceptions**. These exceptions are known as **User defined or Custom exceptions**.
- An exception can be created by extending java **Exception** class.
- No methods are required.
- Constructor can be added as per requirement
- `toString()` function can be overridden , to display customized message.

```
class MyException extends Exception
{
    private int ex;
    MyException(int a)
    {
        ex=a;
    }
    public String toString()
    {
        return "MyException[" + ex +"] is less than zero";
    }
}
```

- Checked exceptions (compile time) are the one which forces the programmer to handle it, without which the program doesn't compile successfully. While unchecked exception (Runtime) doesn't get checked during compilation.
- **"Throws keyword"** is mainly used for handling checked exception as using throws we can declare multiple exceptions in one go
- When we are throwing any exception in a method and not handling it, then we need to use **throws** keyword in method signature to let caller program know the exceptions that might be thrown by the method.
- The caller method might handle these exceptions or propagate it to its caller method using throws keyword.
- We can provide multiple exceptions in the throws clause and it can be used with main() method also.

Example of throws

OBJECT

```
import java.io.IOException;

class Testthrows1 {

    void m() throws IOException {
        throw new IOException("device error");//checked exception
    }

    void n() throws IOException {
        m();
    }

    void p() {
        try {
            n();
        } catch (Exception e) { System.out.println("exception handled"); }
    }

    public static void main(String args[]) {
        Testthrows1 obj = new Testthrows1();
        obj.p();
        System.out.println("normal flow...");
    }
}
```

There are two cases:

Case1: You caught the exception i.e. handle the exception using try/catch. This can be used for critical situations. the code will be executed fine whether exception occurs during the program or not.

Case2: You declare the exception i.e. specifying throws with the method. This can be used when chances of occurring exception is very low. If exception does not occur, the code will be executed fine. If exception occurs, an exception will be thrown at runtime because throws does not handle the exception.

- The exception that is caught by catch block can be rethrown from catch block
- when you declare one or more exception types in a catch clause, and rethrow the exception handled by this catch block, the compiler verifies that the type of the rethrown exception meets the following conditions:
 - ◆ The try block is able to throw it.
 - ◆ There are no other preceding catch blocks that can handle it.
 - ◆ It is a subtype or supertype of one of the catch clause's exception parameters.



- A single catch block can handle more than one type of exception. This feature can reduce code duplication and lessen the temptation to catch an overly broad exception.
- The catch clause specifies the types of exceptions that the block can handle, and each exception type is separated with a vertical bar (|).

```
catch (IOException | SQLException ex) {  
    logger.log(ex);  
    throw ex;  
}
```



- A resource is an object that must be closed once your program is done using it. For example a File resource or JDBC resource for database connection or a Socket connection resource.
- Before Java 7, there was no auto resource management and we should explicitly close the resource once our work is done with it. Usually, it was done in the finally block of a try-catch statement.
- This approach used to cause memory leaks and performance hit when we forgot to close the resource. 🖱️

- Before Java 7

```
try{  
    //open resources like File, Database connection,  
    Sockets etc  
} catch (FileNotFoundException e) {  
    // Exception handling like FileNotFoundException,  
    IOException etc  
}finally{  
    // close resources  
}
```

- Java 7 try with resources implementation

```
try{// open resources here){  
    // use resources  
} catch (FileNotFoundException e) {  
    // exception handling  
}  
// resources are closed as soon as try-catch block is  
executed.
```

Example

OBJECT

```
try(BufferedReader br = new BufferedReader(new FileReader("d:\\myFile.txt"))
{
    String str;
    while((str=br.readLine())!=null)
    {
        System.out.println(str);
    }
} catch(IOException ie)
{
    System.out.println("exception");
}
```

- Note : we do not need to explicitly call `close()` method to close **BufferedReader** stream.
- **try statement** contains a parenthesis in which one or more resources are declared . Any object that implements `java.lang.AutoCloseable` or `java.io.Closeable`, can be passed as a parameter to **try statement**.

Imp methods of Throwable

OBJECT

- **public String getMessage()**
Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.
- **public Throwable getCause()**
Returns the cause of the exception as represented by a Throwable object.
- **public String toString()**
Returns the name of the class concatenated with the result of `getMessage()`.
- **public void printStackTrace()**
Prints the result of `toString()` along with the stack trace to `System.err`, the error output stream.
- **public StackTraceElement[] getStackTrace()**
Returns an array containing each element on the stack trace. The element at index 0 represents the top of the call stack, and the last element in the array represents the method at the bottom of the call stack.
- **public Throwable fillInStackTrace()**
Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

- **Separating Error-Handling Code from "Regular" Code :**

In traditional programming, error detection, reporting, and handling often lead to confusing spaghetti code. Exceptions provide the means to separate the details of what to do when something out of the ordinary happens from the main logic of a program

- **Propagating Errors Up the Call Stack**

ability to propagate error reporting up the call stack of methods ➡

- **Grouping and Differentiating Error Types**

you can create groups of exceptions and handle exceptions in a general fashion, or you can use the specific exception type to differentiate exceptions and handle exceptions in an exact fashion.